IronCircle

IR Expert

Certification

This is to acknowledge that

Reginald Edgar

Has successfully completed all requirements and criteria for

IR Expert

Issue Date	Expiry Date
Dec 11, 2025	Dec 11, 2026

Roman Senko
VP Learning

Certification ID. 1012824

IronCircle

IR Expert

Certification info

In this certification, you'll tackle advanced simulations in the TDX-Arena system, covering key course topics.
It aims to demonstrate your incident response skills via hands-on experience, enabling you to independently manage cyber incidents.
Completion requires crafting a detailed Incident Response report, showcasing your ability to handle and mitigate cybersecurity threats effectively, and proving your readiness for the real-world cybersecurity landscape.

Certification ID. 1012824

Final Assessment Report Submission

Student: Reginald Edgar

Case: One of US

Date: 2025-11-29

Executive Summary

This case involved identifying a malicious file hidden among a large directory of suspicious executable files on a Linux system. The challenge environment restricted traditional antivirus tools such as clamscan, freshclam, and service control, and the provided ClamAV-UI web interface allowed uploading only a single file at a time, making bulk analysis inefficient. Initial attempts to enhance the site's upload functionality through browser developer tools did not result in a workable solution.

Due to these restrictions, I shifted to a forensic methodology based on static analysis. After confirming all files had unique hashes and no signature tools such as sigtool or strings were available, I used the Linux 'file' command to analyze each file's internal format. One file differed from the rest by being identified as an MS-DOS executable, while all others were PE32 executables. This anomaly revealed the malicious file, from which I extracted the required hash signature, completing the investigation successfully.

Findings and Analysis

Malicious File

Finding Details: file.exe (MS-DOS executable)

Description: This file differed from the PE32 executables in the directory and was identified as the malicious sample.

Hash

Finding Details: SHA-256 hash of the malicious file

Description: After identification, the file was scanned with the clamav website and the hash was provided as the flag.

File Attribute

Finding Details: Executable Type

Description: The 'file' command identified one file as MS-DOS format while all others were PE32, making it stand out.

Technique

Finding Details: Static File Type Analysis

Description: Determining the file format using embedded magic numbers allowed isolation of the malicious file without AV tools.

Methodology

Tools and Technologies Used

ClamAV-UI: Initially attempted for scanning; limited to one-file uploads and unsuitable for bulk analysis.

Browser Developer Tools (F12): Used to modify HTML input attributes in an attempt to upload entire directories.

sha256sum: Used to verify that all files had unique hashes, confirming no trivial hash-based outlier.

file: Primary tool used to identify differences in executable file types through magic number analysis.

Investigation Process

- Attempted to use ClamAV-UI, but it only allowed selecting one file at a time.

- Tested wildcard and multi-select methods inside the file dialog; these were blocked.

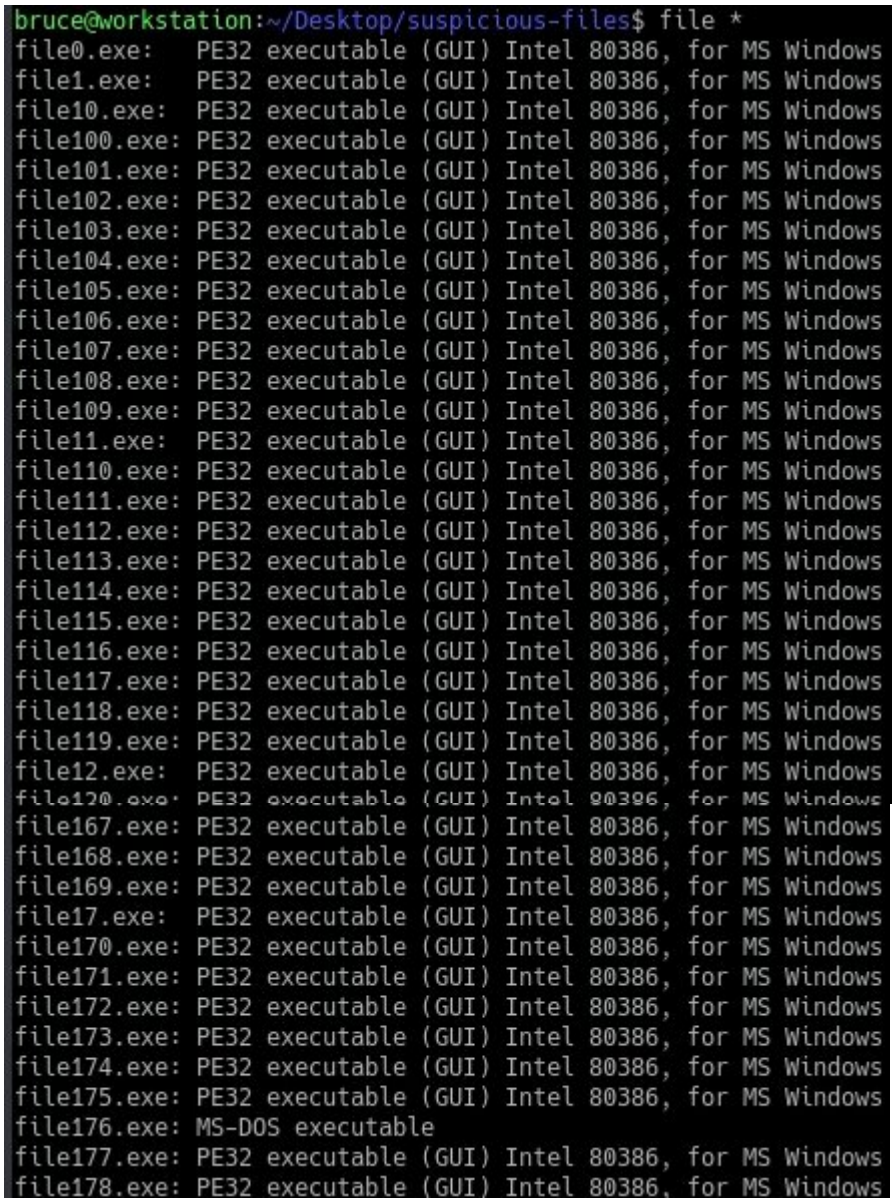
- Checked installed ClamAV packages and attempted to use clamscan, freshclam, and daemon services—none were permitted.

- Used browser developer tools to modify the <input> element to allow uploading entire directories (multiple, webkitdirectory), but site failed to process them.

- Pivoted to static analysis approaches due to environmental restrictions.

- Computed SHA-256 hashes of all files; all differed, indicating hashing would not reveal the malicious file.

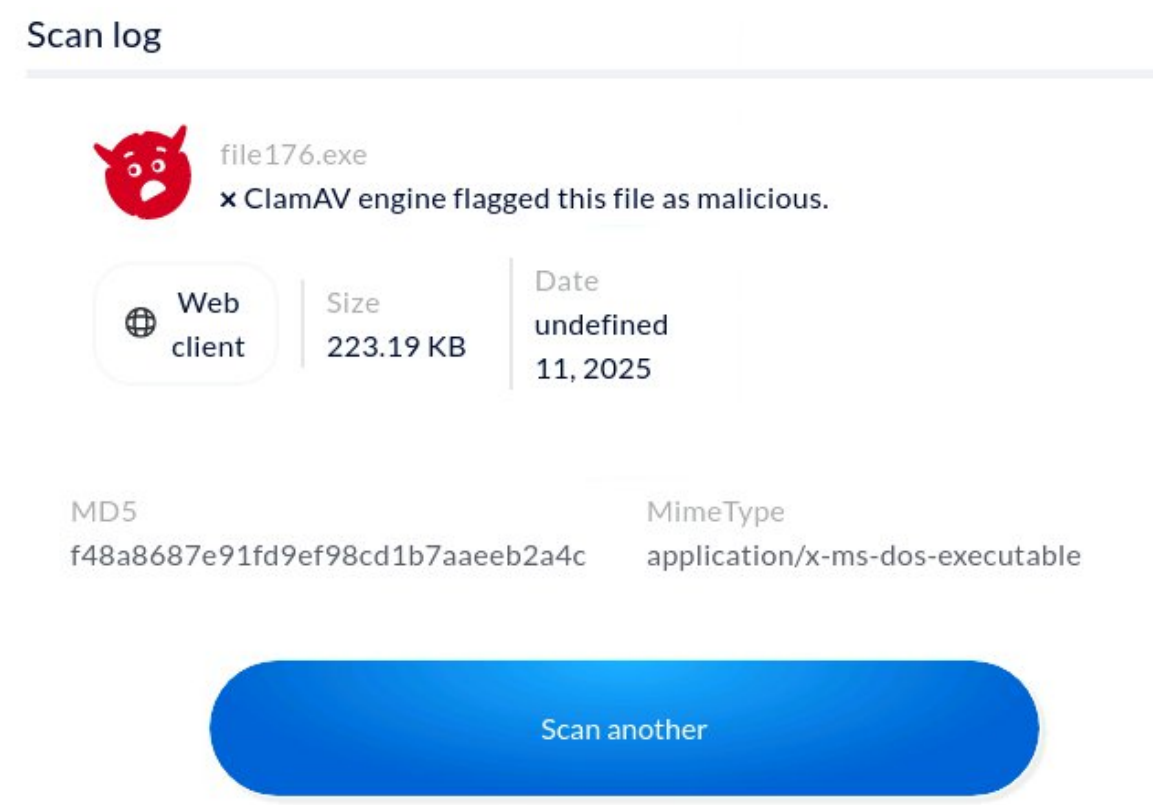
- Used the 'file' command to analyze binary structure of all executables.



- Found one file identified as an MS-DOS executable while the others were PE32.

- Confirmed the outlier as the malicious file and extracted its hash.

```
bruce@workstation:~/Desktop/suspicious-files$ md5sum file176.exe
f48a8687e91fd9ef98cd1b7aaeeb2a4c  file176.exe
bruce@workstation:~/Desktop/suspicious-files$
```



Recommendations

- Ensure all workstations run centrally managed antivirus/EDR with real-time protection and behavior-based detection.

- Enforce continuous signature and engine updates across all endpoints to prevent signature lag and improve detection.

- Deploy an Endpoint Detection & Response (EDR) platform to monitor suspicious processes, unauthorized file creation, and persistence mechanisms.

- Implement application allowlisting (e.g., AppLocker or WDAC) to prevent execution of unapproved or unknown binaries.

- Enforce least-privilege access by removing unnecessary local administrator rights from employee workstations.

- Deploy centralized logging (SIEM) with Sysmon or equivalent endpoint telemetry for visibility into process execution and file changes.

- Conduct regular threat hunts and host integrity checks to detect secondary payloads or hidden malware components.

- Provide ongoing security awareness training to reduce the risk of phishing and inadvertent execution of malicious files.

- Maintain hardened workstation images and automate rapid re-imaging procedures for compromised systems.

- Strengthen network-level defenses, including DNS filtering, outbound connection monitoring, and IDS/IPS signatures to detect malicious C2 activity.

Final Assessment Report Submission

Student: Reginald Edgar

Case: Pigs Rules

Date: 12/11/25

Executive Summary

This case involved analyzing live network traffic to identify malicious inbound activity and configuring the Snort IDS to detect the threat.

Using tcpdump, normal traffic was filtered out to expose a persistent SYN scan. Snort was corrected to log properly and the rule successfully detected the attack.

Findings and Analysis

Finding: IP Address
Finding Details: 172.29.0.1
Description: Source of large-scale malicious SYN traffic.

Finding: Packet Attribute
Finding Details: SYN flag only, len 0, window 1024
Description: Attributes indicated a stealth SYN port scan.

Finding: Attack
Finding Details: TCP SYN Stealth Port Scan
Description: Reconnaissance technique used by attackers.

Finding: Port
Finding Details: Multiple sequential ports
Description: High-volume scan across many ports.

Finding: Process / Tool Behavior
Finding Details: Snort misconfigured, outputting to console, not ingested by snorby.
Description: Snorby not receiving alerts until corrected.

Methodology

Tools and Technologies Used

• tcpdump – for packet capture and filtering.

• Snort – IDS for alerting and rule testing.

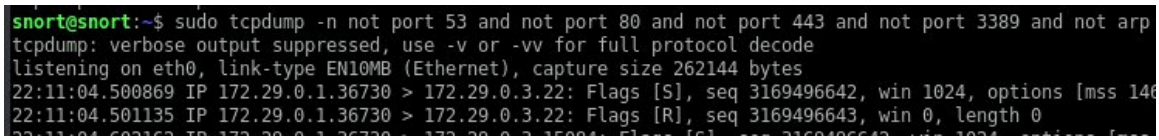
• Snorby – visualization of IDS alerts.

• Linux terminal – environment for analysis.

Investigation Process

• Monitored live traffic using tcpdump.

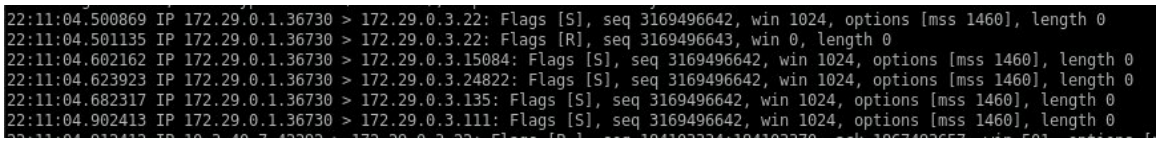
• Filtered normal ports (80, 53, 443, 3389) to isolate anomalies.



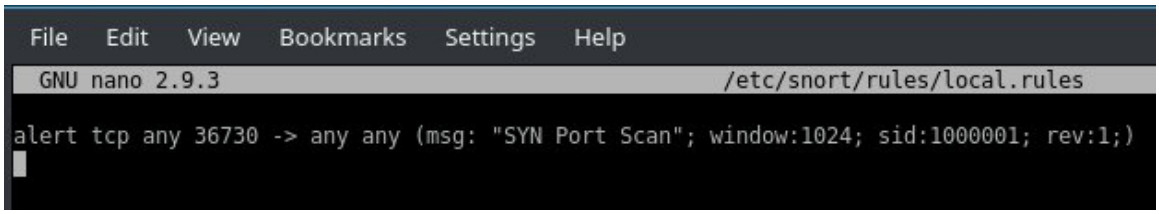
• Identified high-volume SYN packets from 172.29.0.1.

• Filtered out noise to confirm no hidden activity.

• Determined SYN port scan as primary attack.

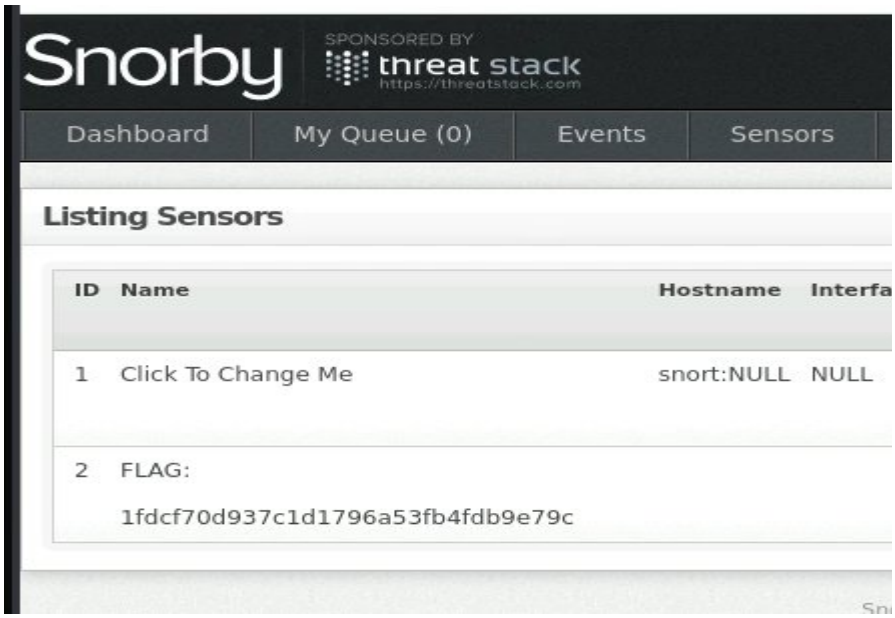


• Created and refined Snort rules based on captured packet attributes.



• Corrected Snort execution to enable unified2 output and Snorby ingestion.

• Final rule deployed: alert tcp any 36730 -> any any (msg:"SYN Port Scan"; window:1024; sid:1000001; rev:1;)



Recommendations

- Implement SYN flood mitigation such as SYN cookies and rate limiting.
- Use firewalls to block repeated malicious connection attempts.
- Restrict inbound traffic to necessary service ports.
- Disable unused services and maintain regular patching.
- Centralize logs via SIEM for correlation and detection.
- Monitor for repetitive SYN patterns or abnormal packet attributes.
- Segment networks to minimize lateral exposure.

Imperial Memory Challenge

Student: Reginald Edgar
Case: Imperial Memory
Date: 12/10/25

Executive Summary

This case involved analyzing a volatile memory dump (`Emperor.vmem`) to uncover clues leading to the extraction of an encrypted archive and the discovery of a hidden flag. The objective required multilayered forensic analysis: string extraction from the memory image, investigation of an embedded `.7z` archive password, extraction and inspection of `suspicious.docx`, and finally discovery of a concealed indicator within the embedded file `secrets.txt`.

The investigation demonstrated a methodical approach: running `strings` on the memory dump to locate references to `gift.7z`, identifying a plaintext password within memory, extracting and analyzing the DOCX contents, and applying multiple hashing and inspection methods to identify the hidden flag. Several investigative avenues were attempted—including XML parsing, metadata review, hex inspection, and searching for flag formats—before identifying the correct flag using the MD5 hash of the embedded file.

The investigation concluded successfully with the identification of the challenge flag, demonstrating proficiency in memory forensics, document forensics, and investigative methodology.

Findings and Analysis

Finding 1 — Process / Data Discovery

Finding Details: Password for `gift.7z` found inside RAM
Description:
Executing `strings` on the memory dump and grepping for `gift.7z` revealed multiple references to the archive and, eventually, the plaintext password. This was the first critical discovery and allowed extraction of `suspicious.docx`. This confirmed RAM as a valid recovery source for user-handled secrets.

Finding 2 — Technique

Finding Details: Use of memory-resident artifacts to recover credentials
Description:
The ability to recover the archive password from RAM reinforced a key incident-response concept: sensitive information frequently persists in volatile memory. This validated the forensics approach taken and demonstrated the effectiveness of memory analysis in recovering credentials and IOCs.

Finding 3 — File Structure Anomaly

Finding Details: Hidden file inside DOCX ZIP container
Description:
Upon unzipping `suspicious.docx`, the directory structure appeared mostly standard. However, an additional plaintext file, `secrets.txt`, existed despite not being referenced by any Word XML relationships. This anomaly suggested intentional concealment—common in CTF challenges—and made the file a high-interest target.

Finding 4 — File Attribute

Finding Details: `secrets.txt` — size: **1389 bytes**
Description:
Inspection showed unusual attributes such as an early Windows timestamp (1980) and the absence of any connection to visible document content. These characteristics reinforced the suspicion that the file contained something of significance and warranted deeper examination.

Finding 5 — Hash

Finding Details: MD5 hash of `secrets.txt`
Description:
After ruling out visible text clues and hidden data via hexdump and XML review, multiple hash functions were tested. The MD5 hash of `secrets.txt` matched the challenge's expected flag format. SHA1 and SHA256 did not. This provided the final actionable flag and concluded the investigation.

Methodology

Tools and Technologies Used

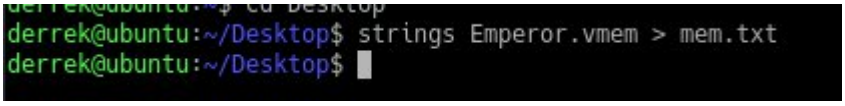
- **strings** — Used to extract printable ASCII data from the memory dump to reveal archived passwords and context.

- **grep** — Used extensively to search for references to files, passwords, flags, and suspicious content.
- **unzip** — Used to enumerate and extract DOCX contents.
- **hexdump** — Used to manually inspect binary content of DOCX internals.
- **md5sum / sha1sum / sha256sum** — Used to calculate hashes of files when searching for possible encoded flags.
- **Linux shell environment** — Provided core forensic tooling without needing additional installations.

Investigation Process

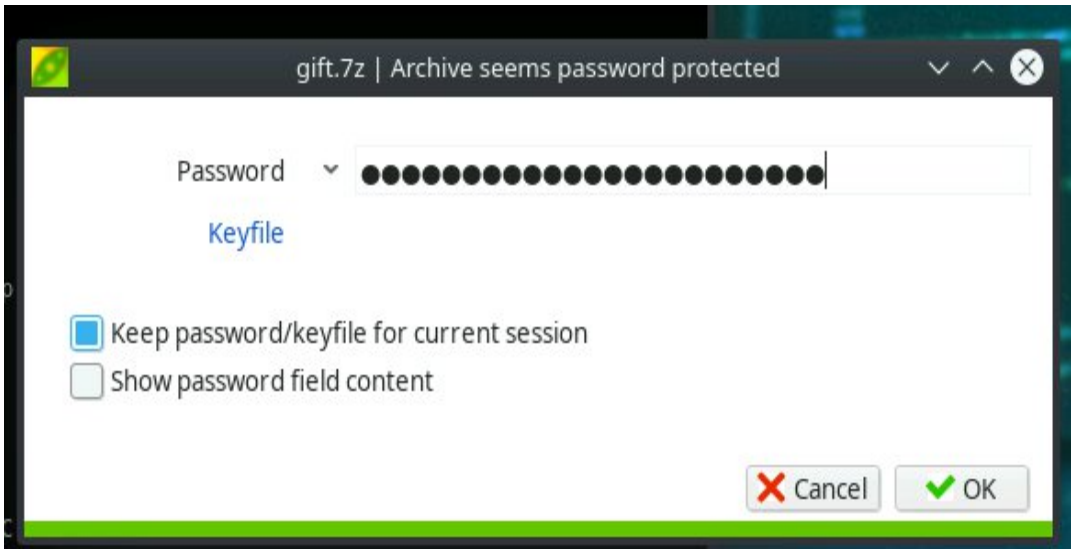
Initial Memory Dump Review

- Executed: `strings Emperor.vmem > mem.txt`
- Purpose: Collect all readable text from RAM for offline searching.
- Contribution: Provided the first lead—references to **gift.7z**.



Targeted Search of RAM Strings

- Commands:
  `grep -i gift mem.txt`
  `grep -i 7z mem.txt`
- Located a plaintext password allowing extraction of the archive.
- Contribution: Directly enabled access to `suspicious.docx`.



Extraction of Encrypted Archive

- Used the recovered password to extract: `suspicious.docx`
- Contribution: Produced the file requiring deeper investigation.

Inspection of DOCX Contents (ZIP Container)

- Commands:
  ```
  unzip suspicious.docx -d sus_docx
  zipinfo -z suspicious.docx
  ```
- Verified folder structure: `_rels`, `docProps`, `word`, and XML files were normal.
- Identified *one abnormal file*: `secrets.txt`.

```
derrek@ubuntu:~/Desktop$ unzip suspicious.docx -d sus_docx
Archive:  suspicious.docx
  inflating: sus_docx/[Content_Types].xml
  inflating: sus_docx/_rels/.rels
  inflating: sus_docx/word/document.xml
  inflating: sus_docx/word/_rels/document.xml.rels
  inflating: sus_docx/word/theme/theme1.xml
  inflating: sus_docx/word/settings.xml
  inflating: sus_docx/word/styles.xml
  inflating: sus_docx/word/webSettings.xml
  inflating: sus_docx/word/fontTable.xml
  inflating: sus_docx/docProps/core.xml
  inflating: sus_docx/docProps/app.xml
  inflating: sus_docx/secrets.txt
```

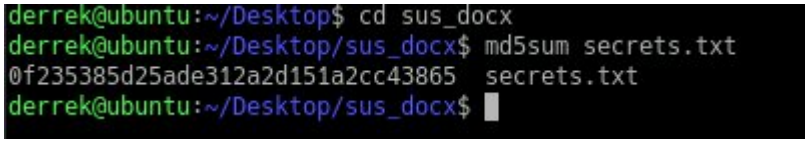
Analysis of secrets.txt

- Reviewed text content regarding “Law of Individuality,” “Probability,” and “Circumstantial Facts.”
- Initially appeared unrelated to flag format.
- Contribution: Established need for deeper non-visual inspection.

Hex and Metadata Inspection

- Command: `hexdump -C secrets.txt`
- No embedded flag or encoded content found.
- Contribution: Eliminated common steganographic possibilities.

Hash Testing

- Calculated multiple hashes of `secrets.txt`:
  `sha1sum secrets.txt` → not valid
  `sha256sum secrets.txt` → not valid
  `md5sum secrets.txt` → **flag found**
- The MD5 value matched expected challenge flag format.
- Contribution: Successful discovery of the required flag.
- 

Recommendations

While this challenge scenario is benign, real-world environments benefit from:

1. Memory Forensic Readiness

- Enable full RAM capture capability for IR teams.
- Maintain training on Volatility and other RAM-analysis tools.
- Configure EDR solutions to detect credential artifacts in memory.

2. IOC-Hunting Enhancements

- Automate scanning of RAM for high-value artifacts (passwords, tokens).
- Maintain hash databases to quickly correlate suspicious files.
- Apply anomaly-detection rules to detect unexpected file formats embedded inside documents.

3. Document Security Hygiene

- Flag documents containing unreferenced ZIP entries (e.g., hidden TXT files).
- Implement automated scanning for embedded scripts, macros, or anomalous metadata.

4. Archive & File Activity Monitoring

- Log interactions with encrypted archives.
- Correlate user actions (opening archives, extracting files) with RAM artifacts to support investigations.